

GEZEL: Your Open Source HDL Solution

ASHWIN GOPINATH

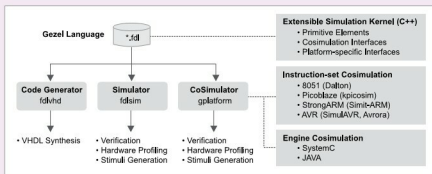
In this edition of *Electronics for You Plus* we have bundled in the DVD a nice little software capable of simulation, co-simulation as well as compiling into VHDL code. Its name is GEZEL and it is distributed for free by Patrick Schaumont and the team.

It is a hardware description language (HDL), which allows you the implementation of a finite-state machine and datapath model. This model can be accurately described as a mathematical abstraction that is sometimes used to design digital logic or computer programs. It consists of states that are essentially used to assign sequences. For example, a program is converted into a state diagram, which shows logical sequencing of the circuit. It is similar, though less powerful, than a Turing machine. But where the Turing machine is unrestricted, the number of states here are dictated by the memory of a finite-state machine.

Highlights of GEZEL

As mentioned earlier, GEZEL is a cycle based HDL relying on the finite-state machine and datapath model. It allows you to add new co-simulation interfaces by utilising user-defined library block extensions in C++. Most of all, since it is open source, we are following EFY's policy of promoting open source solutions for all your electronic needs.

GEZEL possesses a scripted environment with easy-to-learn HDL. It allows you to co-simulate using embedded cores of multiple vendors, be it ARM, Intel 8051 and/or ATMEL



Flowchart depicting GEZEL

AVR. Presence of a code generator for synthesisable VHDL code would be extremely useful for first-timers, saving them a lot of time and effort from having to port the circuit on to a separate code simulator. If you fancy learning HDL and are just starting out on the journey, do try GEZEL and let us know your experience with it.

The GEZEL structure

GEZEL has three main tools that it utilises to perform its functions.

GEZEL standalone simulation tool. This is a nice version of an open source simulator. Based on a cycle-true simulation algorithm,

the simulator, called fdlsm, has an evaluate phase and a register-update phase. Evaluate the condition; if true, update register, and if not, proceed accordingly. Simulation output can be modified with the help of simulation directives, namely, display, control and trace. Command line for fdlsm is:

```
fdlsm [-d] [<design.fdl>] <cyclecount>
```

(The -d is a debug flag, and when enabled, the simulator provides a more detailed account of the activities.)

GEZEL co-simulation engine.

Three methods of simulation

Use command line. The following program, when run, executes three clock styles.

fdlsm listing9.fdl 3

Use standard input. Using standard input allows you to pipe the design into the simulator.

cat listing9.fdl | fdlsm 3

Use scripting feature of the shell. The only caveat here is that you must provide the location of fdlsm in the code.

listing10.fdl 3

Called gplatform, it helps in co-simulating GEZEL designs with instruction-set simulators. Advantage of such co-simulation efforts is a design that can include co-processors working on a host of different functions. gplatform supports co-simulation with multiple ARM cores and/or 8051 microcontrollers.

If you are wondering what a co-simulation engine does, then do not worry. It is merely an abstracted version of the real hardware/software interface. Another cycle-true model, there are many forms of co-simulation interfaces, depending on